Rate limiter design

Shared budget of 10,000 requests per second across 5 servers, with a mid-design revision that removes Redis.

Part A — Binary search

Time complexity: O(log n).

Part B — Shared 10k requests/second

The limit is one shared budget of 10,000 requests/second for the whole fleet. Five independent limiters at 10k would allow 50k. Five hard caps at 2k stay under the ceiling only by rejecting traffic that another server could have taken.

Choice: token bucket, with capacity 10,000 and refill 10,000 tokens/second.

The requirement is a sustained rate, and those two numbers are that rate. An empty bucket admits at most one second of traffic immediately, then settles at 10k/s. State is a token count plus a last-refill timestamp, so each decision is O(1).

A sliding-window log would keep on the order of 10k timestamps per second for a single global key. A sliding-window counter is cheaper and is the right tool when the contract is “N events in every period of length T” (billing, quota fairness). Overload protection maps directly onto a refill rate.

Shared counter

Each app server runs one atomic update against a shared key `rl:global`:

1. Read `tokens` and `last_refill_ms`. A missing key starts full (10,000).
2. `tokens = min(10000, tokens + (now - last_refill_ms) * 10)`.
3. If `tokens >= 1`, decrement and allow; otherwise deny.
4. Write the new count and timestamp.

One Redis primary handles 10k of these scripts per second on a single key. To keep the key off the critical path, each server leases a small batch (about 50 tokens, expiring in about 100ms) and spends it from memory. Leased tokens are already removed from the global bucket, so admissions stay at or under 10k. Expiry returns anything unspent so an idle server does not strand budget.

A deny responds with `429` and `Retry-After` set from the time until the next token. At full refill that interval is sub-millisecond, so a few milliseconds is a practical client hint.

If Redis cannot be reached, each server falls back to its own in-memory bucket at 2,000 tokens/second, which still sums to 10k.

Part C — Revision: no Redis

The interruption is: “Actually, we can’t use Redis.”

The bucket parameters stay: capacity 10,000, refill 10,000/s, and a deny still means fewer than one token remains. Redis was only the single atomic counter. Two placements keep that counter without it.

Single edge proxy

If one proxy already sees every request, the bucket moves there. nginx `limit_req` (leaky bucket: `rate=10000r/s`, `burst=0`) or an Envoy local rate limit enforces the same ceiling in front of all five upstreams. The app processes stay stateless. A proxy restart is the only new failure domain. While it is down, the partitioned fallback below applies.

Direct to the five servers

If traffic hits the five servers directly, each process keeps an in-memory token bucket of capacity 2,000 refilled at 2,000/s. The sum of the ceilings is 10,000, so the fleet cannot exceed the budget.

Spare capacity is recovered by borrowing. A server that is denying asks a peer for a lease of about 100 tokens over the next 100ms. The lender deducts that lease from its own bucket immediately, so spendable tokens across the five processes still add up to at most 10k. Unreachable peers leave the static 2k split in place, which is the same safe fallback as a Redis outage.

What stays the same

- Algorithm: token bucket, not a sliding window.
- Global ceiling: 10,000 admissions per second.
- Safe degraded mode: five local buckets at 2,000/s.
- Client signal: 429 with Retry-After.